

✓ Point of Sale System

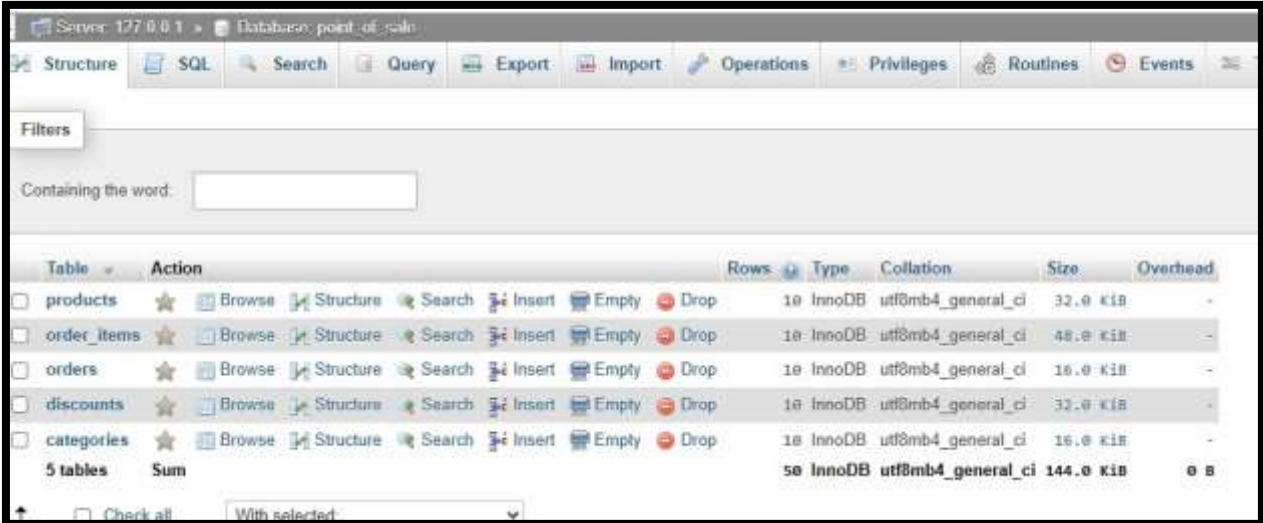
1. Introduction

In this lab, we designed a Point of Sale (POS) database system. The database simulates a small retail environment where products are categorized, customers place orders, and discounts are applied. The goal is to practice creating tables, defining relationships, inserting sample data, and running queries to extract useful information.

2. Database Structure

The database consists of the following tables:

- **Roles** – Defines user roles (Admin, Customer, Salesman).
- **Categories** – Groups products into categories such as Beverages, Snacks, Dairy, etc.
- **Products** – Stores product details including name, price, and category.
- **Orders** – Records customer orders with date and customer name.
- **Order Items** – Links products to orders with quantities.
- **Discounts** – Stores discount percentages for products.



The screenshot shows the MySQL Workbench interface for a database named 'point_of_sale'. The 'Structure' tab is active, displaying a list of tables: products, order_items, orders, discounts, and categories. Each table has a set of actions (Browse, Structure, Search, Insert, Empty, Drop) and columns for Rows, Type, Collation, Size, and Overhead. The 'products' table has 10 rows, 'order_items' has 10 rows, 'orders' has 10 rows, 'discounts' has 10 rows, and 'categories' has 10 rows. The total size of the database is 144.0 KiB.

Table	Action	Rows	Type	Collation	Size	Overhead
products	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_general_ci	32.0 KiB	-
order_items	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_general_ci	48.0 KiB	-
orders	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_general_ci	16.0 KiB	-
discounts	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_general_ci	32.0 KiB	-
categories	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_general_ci	16.0 KiB	-
5 tables	Sum	50	InnoDB	utf8mb4_general_ci	144.0 KiB	0 B

3. Data Insertion

Sample data was inserted into each table:

- **Categories:** 10 categories (Beverages, Snacks, Dairy, Bakery, Fruits, Vegetables, Frozen, Meat, Seafood, Grains).

Point Of Sale

- **Products:** 10 products (Coke, Pepsi, Chips, Milk, Bread, Apple, Carrot, Ice Cream, Chicken, Fish).

	product_id	name	price	category_id
☐ Edit Copy Delete	1	Coke	100.00	1
☐ Edit Copy Delete	2	Pepsi	90.00	1
☐ Edit Copy Delete	3	Chips	50.00	2
☐ Edit Copy Delete	4	Milk	120.00	3
☐ Edit Copy Delete	5	Bread	80.00	4
☐ Edit Copy Delete	6	Apple	150.00	5
☐ Edit Copy Delete	7	Carrot	70.00	6
☐ Edit Copy Delete	8	Ice Cream	200.00	7
☐ Edit Copy Delete	9	Chicken	500.00	8
☐ Edit Copy Delete	10	Fish	600.00	9

Orders and Order Items

- **Orders:** 10 orders placed by customers (Ali, Ahmed, Sara, Zain, Ayesha, Usman, Hina, Bilal, Fatima, Omar).
- **Order Items:** Each order linked with products and quantities.

Point Of Sale

	order_id	order_date	customer_name
☐ Edit Copy Delete	1	2026-01-01	Ali
☐ Edit Copy Delete	2	2026-01-02	Ahmed
☐ Edit Copy Delete	3	2026-01-03	Sara
☐ Edit Copy Delete	4	2026-01-04	Zain
☐ Edit Copy Delete	5	2026-01-05	Ayesha
☐ Edit Copy Delete	6	2026-01-06	Usman
☐ Edit Copy Delete	7	2026-01-07	Hina
☐ Edit Copy Delete	8	2026-01-08	Bilal
☐ Edit Copy Delete	9	2026-01-09	Fatima
☐ Edit Copy Delete	10	2026-01-10	Omar

- **Discounts:** Discounts are applied to products. Each products can have a discount percentage.

Point Of Sale

Server: 127.0.0.1 Database: point of sale Table: discounts

Showing rows 0 - 9 (10 total, Query took 0.0004 seconds.)

`SELECT * FROM `discounts``

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

		discount_id	product_id	discount_percent
☐	Edit Copy Delete	1	1	10
☐	Edit Copy Delete	2	2	5
☐	Edit Copy Delete	3	3	15
☐	Edit Copy Delete	4	4	8
☐	Edit Copy Delete	5	5	12
☐	Edit Copy Delete	6	6	7
☐	Edit Copy Delete	7	7	6
☐	Edit Copy Delete	8	8	20
☐	Edit Copy Delete	9	9	10
☐	Edit Copy Delete	10	10	5

Check all | With selected: Edit Copy Delete Export

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 item_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 order_id	int(11)			Yes	NULL			Change Drop More
☐	3 product_id	int(11)			Yes	NULL			Change Drop More
☐	4 quantity	int(11)			Yes	NULL			Change Drop More

Check all | With selected: Browse Change Drop Primary Unique Index Spatial Fulltext

Add to central columns Remove from central columns

Print Propose table structure Track table Move columns Normalize

Add 1 column(s) after quantity Go

Indexes

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	item_id	10	A	No	
Edit Rename Drop	order_id	BTREE	No	No	order_id	10	A	Yes	
Edit Rename Drop	product_id	BTREE	No	No	product_id	10	A	Yes	

Create an index on 1 columns Go

The screenshot displays the 'Table structure' tab for a table named 'Orders'. The table has four columns: 'product_id' (int(11), AUTO_INCREMENT, primary key), 'name' (varchar(100)), 'price' (decimal(10,2)), and 'category_id' (int(11)). Below the table structure, there are various actions like 'Change', 'Drop', and 'More'. At the bottom, there is a section for 'Indexes' showing two indexes: 'PRIMARY' on 'product_id' and another on 'category_id'.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	product_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	name	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
3	price	decimal(10,2)			Yes	NULL			Change Drop More
4	category_id	int(11)			Yes	NULL			Change Drop More

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	product_id	10	A	No	
Edit Rename Drop	category_id	BTREE	No	No	category_id	10	A	Yes	

4. Queries and Results

- **Total Orders**

This query shows the total number of orders placed.

The screenshot shows a green banner with the text 'Your SQL query has been executed successfully.' Below it, the SQL query is displayed: `-- Total orders SELECT COUNT(*) AS Total_Orders FROM Orders;`

- **Total Sales per Product**

This query calculates how many units of each product were sold.

Point Of Sale

Showing rows 0 - 9 (10 total, Query took 0.0007 seconds)

```
-- Total sales per product SELECT p.name, SUM(oi.quantity) AS total_sold FROM Order_Items oi JOIN Products p ON oi.product_id = p.product_id GROUP BY p.name;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

name	total_sold
Apple	3
Bread	1
Carrot	4
Chicken	2
Chips	5
Coke	2
Fish	1
Ice-Cream	1
Milk	2
Pepsi	1

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Activate Windows

- **Orders with Customer Names**

Displays all orders along with customer names and dates.

Showing rows 0 - 9 (10 total, Query took 0.0004 seconds)

```
-- Orders with customer names SELECT order_id, customer_name, order_date FROM Orders;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	order_id	customer_name	order_date
☐ Edit Copy Delete	1	Ali	2026-01-01
☐ Edit Copy Delete	2	Ahmed	2026-01-02
☐ Edit Copy Delete	3	Sara	2026-01-03
☐ Edit Copy Delete	4	Zain	2026-01-04
☐ Edit Copy Delete	5	Ayesha	2026-01-05
☐ Edit Copy Delete	6	Usman	2026-01-06
☐ Edit Copy Delete	7	Hina	2026-01-07
☐ Edit Copy Delete	8	Bilal	2026-01-08
☐ Edit Copy Delete	9	Fatima	2026-01-09
☐ Edit Copy Delete	10	Omar	2026-01-10

☐ Check all | With selected: Edit Copy Delete Export

Activate Windows
Go to Settings to activate Windows.

Point Of Sale

- **Products with Discounts**

Shows products with their respective discount percentages.



The screenshot shows a database query result in a web interface. At the top, a status bar indicates 'Showing rows 0 - 9 (10 total, Query took 0.0006 seconds.)'. Below this, the query text is displayed: 'Products with discounts SELECT p.name, d.discount_percent FROM Products p JOIN Discounts d ON p.product_id = d.product_id;'. There are links for '[Edit inline]', '[Edit]', and '[Create PHP code]'. Below the query, there are controls for 'Show all', 'Number of rows' (set to 25), 'Filter rows' (with a search box), and 'Sort by key' (set to None). An 'Extra options' button is also present. The main table has two columns: 'name' and 'discount_percent'. The data rows are as follows:

name	discount_percent
Coke	10
Pepsi	5
Chips	15
Milk	8
Bread	12
Apple	7
Carrot	6
Ice Cream	20
Chicken	18
Fish	5

These queries demonstrate how the POS database can provide insights into sales and customer activity.

5. ER Diagram Explanation

The Entity-Relationship (ER) Diagram visually represents the structure of the database:

- **Roles:** Independent table, not directly linked to others in this lab.
- **Categories → Products:** One category can have many products (1-to-many relationship).
- **Products → Order_Items:** A product can appear in many order items (1-to-many).
- **Orders → Order_Items:** One order can contain multiple items (1-to-many).
- **Products → Discounts:** Each product can have one discount (1-to-1 relationship in this design).

5.1. ER Diagram (Conceptual Description)

Entities: Roles, Categories, Products, Orders, Order_Items, Discounts.

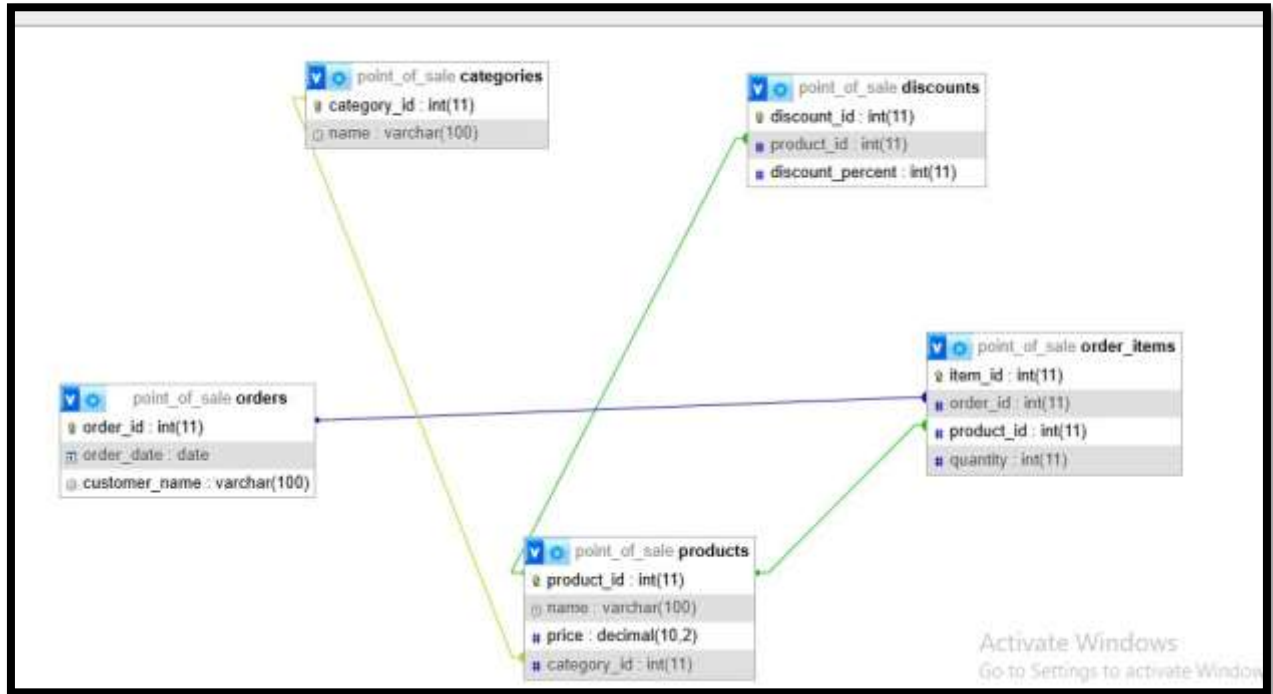
Relationships:

- **Categories → Products (1:N)**

Point Of Sale

- **Orders** → Order_Items (1:N)
- **Products** → Order_Items (1:N)
- **Products** → Discounts (1:1)

This diagram shows how data flows between customers, orders, products, and discounts in the POS system.



Exporting tables from "point_of_sale" database

Export templates:

New template:

Template name:

Existing templates:

Template:

Export method:

☒ Quick - display only the minimal options

☐ Custom - display all possible options

Format:

Activate Windows
Go to Settings to activate Windows

6. Conclusion

This lab successfully demonstrates how to build a Point of Sale database with proper relationships between tables. The queries provide meaningful business insights such as total orders, product sales, customer details, and discounts. The ER diagram further clarifies the logical structure of the system, making it easier to understand and extend in future projects.
